Advanced Programming Proposal Group 13

Backgammon

Kilian Littan 666855kl
Alexandre Careme 697566ac
Sven Krijnsen 702904fk
Diederick Folbert 693584df

7th September 2026

1 Proposal

We propose to implement the board game **Backgammon**. We will begin by describing the game and its rules, followed by an explanation of how we intend to implement AI opponents. Finally, we will discuss several possible extensions that can be used to reduce or improve the complexity of the game, making the implementation more suitable for the requirements of the assignment.

2 Description Backgammon

Backgammon is a board game blending strategy and chance, with roots tracing back to the Roman Empire. It reached its modern form in 18th-century England, with the last major change being the doubling cube, introduced in a New York game club in the 1920s. The modern rules were codified in 1931 by the Card and Backgammon Committee of the New York Racquet and Tennis Club, with minor revisions in 1970. In this standard version one round is won by the player who first manages to get all of his checkers of the board.

It is a **two player zero sum game** that is **non-deterministic** with **perfect information**. Moreover, the **state**, **action**, and **turn spaces** are **discrete**.

3 Backgammon Rules

Adapted from the attached instructions of a Philos-Spiele Backgammon Set and Paul Magriel's Rules of Backgammon (12 December 1999).

Game elements

1 Backgammon board, 15 white checkers and 15 black checkers (in the form of discs),and 4 regular six-sided dice(1 six-sided doubling die for extension)

Terminology

Each player has an outer board and a home board (also called the inner board).

Bar The halfway line separating each player's outer board from their home board.

Point One of the 24 triangles on the board — the 24 possible positions for the checkers.

Aim of the game

The aim of the game is to bring your checkers into your home board and then bear them off (remove from the board). The first player to bear off all of their checkers wins.

Basic set-up

The starting position of the checkers is shown in Figure 1. White plays from the top right to the top left, then to the bottom left and from there to the right into their home board, i.e. anti-clockwise. Black moves in the other direction, clockwise. These directions must be adhered to; moves in the opposite direction are not permitted.

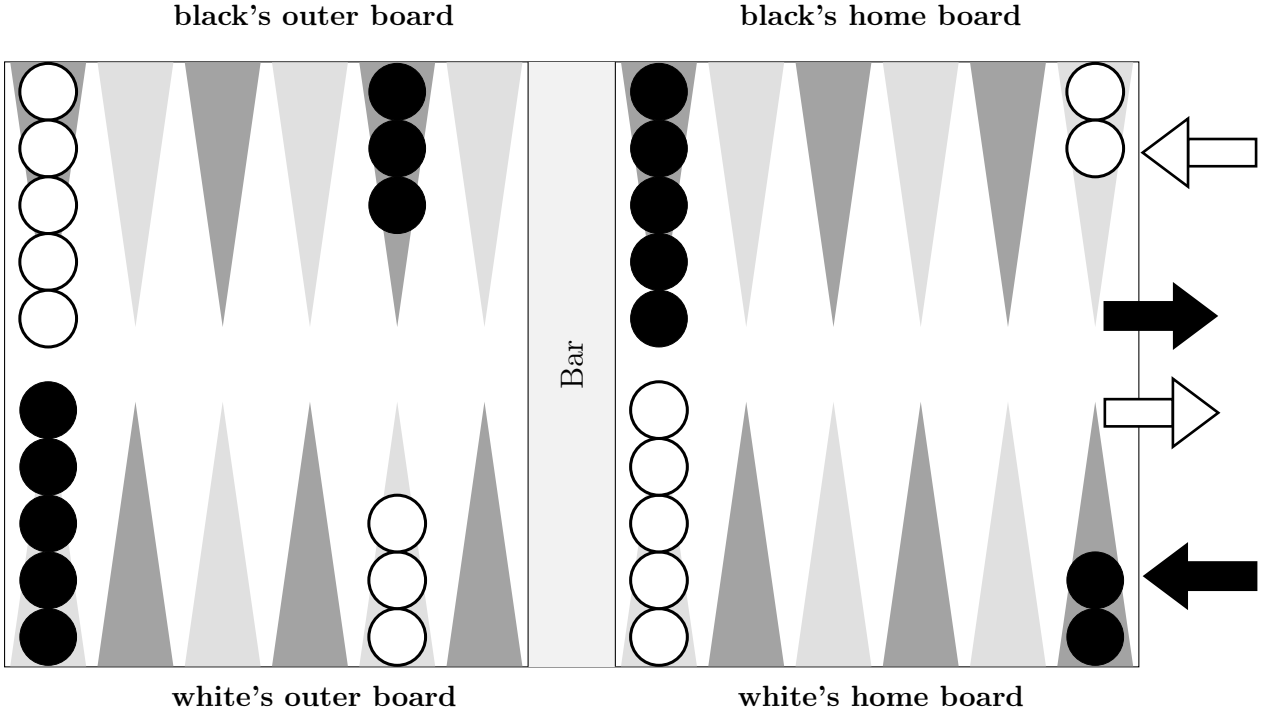


Figure 1: Starting position of the backgammon board. White moves anti-clockwise (from the upper right, down the left side, into the lower-right home board); black moves clockwise.

Start

To start the game, each player throws one die to determine who moves first. The player with the higher number begins. If both players throw the same number, they roll again until one has the higher score. The first move uses the combined total of both dice already thrown.

Throwing the dice

After the opening roll, each turn is made with two dice, with players alternating turns. Each player must throw the dice into the board section on their own right-hand side.

Move

Checkers are moved by the numbers shown on the dice. A player may choose between two ways of moving: moving a single checker using both numbers in sequence, or moving two different checkers, one number each. If a single checker is moved using both numbers, they may not simply be added together — the move must be split into two separate steps, one per die, with a legal intermediate stop on a POINT. If that intermediate POINT is occupied by two or more of

the opponent's checkers, the move is illegal. The player may choose the order in which the two dice are played.
Illegal moves will be impossible to be played by the game interface design.

Example Figure 2. A throw of 2 and 5 lets the player move one checker either 2-then-5 or 5-then-2 points, or move two different checkers 2 and 5 points respectively.

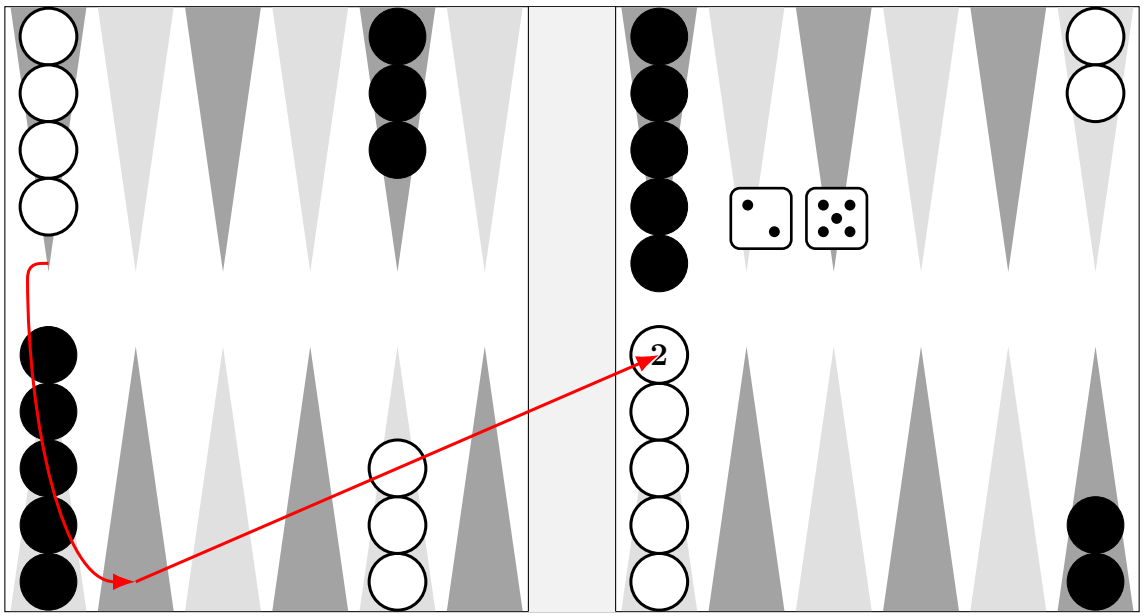


Figure 2: Position after a throw of 2 and 5, with white playing a single checker from the 13 point to the 6 point via the 11 point (i.e. 2 then 5).

Rules for moving

A POINT may be occupied by any number of checkers, except that no player may land on a POINT held by two or more of the opponent's checkers. Both dice must be played if legally possible. If only one checker can be moved, the higher of the two numbers must be used; the lower number is forfeited. It is not legal to play one number in such a way as to make the other number impossible to play. If neither number can be played, the turn is forfeited entirely and passes to the opponent.

Capturing checkers

If only one checker occupies a POINT, it can be hit by the opponent landing on that POINT, either directly or via an intermediate stop. The hit checker is then placed on the bar.

Re-entering from the bar

A player with one or more checkers on the bar may not move any other checker until all of them have re-entered play, in the opponent's home board.

Rules of re-entry. A checker on the bar may only re-enter on an open POINT in the opponent's home board corresponding to the number rolled. If that POINT is held by a single opposing checker, it is hit and sent to the bar. If no legal entry is possible, the move and turn are forfeited.

Double

When a double is thrown, the player moves twice the number shown — in effect, four moves of that number instead of two.

Example. A throw of double 2s gives the player four 2's to play, in any combination: one checker four times, one checker three times and another once, or two checkers twice each.

Bearing off

Bearing off may only begin once all 15 of a player's checkers are in their own home board. Each player then clears checkers off the board by throwing the numbers needed to carry them past the edge of the board. Part or all of a throw may instead be used to bring other checkers further into the home board rather than bearing off — a useful option if the opponent still has a checker on the bar that could hit and send checkers back to the bar. If a checker is hit during bearing off, the player must bring it back into play and return it to their own home board before bearing off can continue.

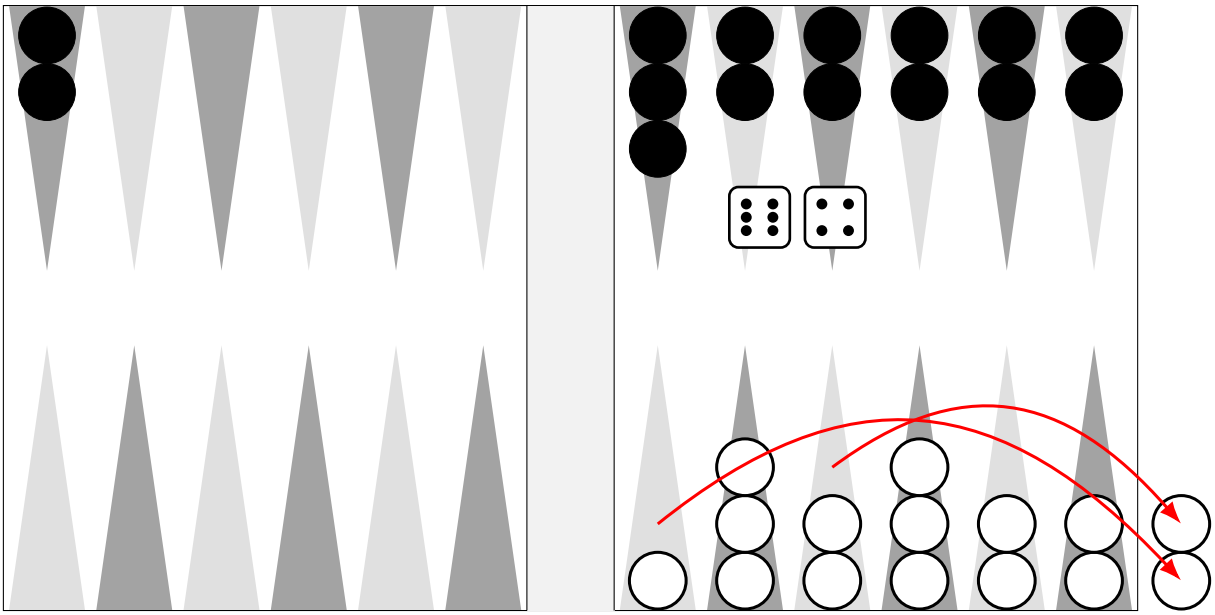


Figure 3: A bearing-off position: black has 13 checkers in its home board but still 2 checkers back in the outer board, so black cannot yet bear off. White, having already brought all 15 checkers home, bears off its first two on a throw of 6 and 4.

End of the game and scoring

Simple win

The winner is whoever bears off all of their checkers first — the opponent may already have borne off several of their own.

Gammon

The winner bears off all of their checkers before the opponent has borne off any at all.

Backgammon

The winner bears off all of their checkers before the opponent has borne off any, while the opponent still has one or more checkers on the bar or in the winner's home board.

Doubling and Tournament rules

Some versions of Backgammon also use a doubling cube, which can double the stakes during play. That variant, along with the various tournament rules, is not covered here, and will only be detailed if we implement it as an extension.

4 AI Opponents

In this section, we describe several AI algorithms that can be implemented as opponents in a one-versus-one Backgammon game. The algorithms differ mainly in how far ahead they look when selecting a move and how they evaluate the resulting board positions.

We begin with a simple one-step-ahead algorithm. This opponent only considers the legal moves available after the dice have already been rolled. Since the dice outcome is known at this point, the algorithm only has to evaluate the board states that can be reached from the current position. Different objective functions can be used to create different styles of play.

We then introduce a more advanced multi-step-ahead algorithm. This algorithm not only considers its own immediate move, but also possible future moves by both players. Since future dice rolls are uncertain, the opponent must take probabilities into account. This leads to an Expectimax or Expectminimax based approach. The advanced opponent builds on the same evaluation functions used by the simple opponent, but applies them to board states several turns into the future.

4.1 One-Step-Ahead AI Opponent

The simplest AI algorithm, corresponding to an easy difficulty level, evaluates only the legal moves available in the current turn. After the dice have been rolled, the outcome is fixed, so the agent can generate all legal moves and evaluate the resulting board states using an objective function. The move with the highest value is then selected. A possible evaluation function is given by:

$$V(s) = \alpha \cdot \text{pointsTillEnd} + \beta \cdot \text{hits} + \gamma \cdot \text{madePoints} + \delta \cdot \text{safeCheckers} + \varepsilon \cdot \text{blots} + \zeta \cdot \text{RNG}$$

where $V(s)$ is the value of the resulting board state s, and $\alpha, \beta, \gamma, \delta, \varepsilon$, and ζ are coefficients that determine the importance of each feature.

The features used in the evaluation function can be defined as follows:

- pointsTillEnd: the total number of points the agent's checkers still need to move before being borne off.
- hits: the number of opponent checkers sent to the bar.
- madePoints: the number of points occupied by two or more of the agent's own checkers.
- safeCheckers: the number of checkers that are not vulnerable to being hit by the opponent.
- blots: the number of single checkers that are exposed to attack.
- RNG: a random value used to introduce variety into the opponent's behaviour.

By changing the coefficients, different playing styles can be created. For example, a positive value of α rewards moves that reduce the total distance the checkers still need to travel. This encourages the agent to move its back checkers forward and make progress toward bearing off. In contrast, a negative value of α could make the agent prefer moves that keep its checkers farther back, which may result in a more defensive style.

Similarly, the coefficient β can be increased to make the agent more aggressive, since it will place more value on hitting opponent checkers. A larger value of γ rewards building strong points, while a larger value of ε can be used as a penalty if exposed blots are considered undesirable. Finally, increasing ζ gives more influence to randomness, making the agent less predictable and allowing for more variation in its moves.

This one-step-ahead approach is computationally inexpensive and therefore suitable for a simple or beginner-level opponent. However, because it only evaluates the immediate consequences of a move, it may fail to recognize long-term opportunities.

4.2 Multi-Step-Ahead AI Agent

Backgammon is a non-deterministic game because future dice rolls are unknown. To create a stronger AI opponent, the agent must therefore consider not only its own current move, but also possible future dice rolls and the opponent's possible responses. This results in a game tree in which decision nodes represent choices made by the players, while chance nodes represent dice outcomes.

A natural approach for this type of problem is the ExpectMinimax algorithm. In a deterministic two-player game, the Minimax algorithm assumes that one player tries to maximise their own evaluation score, while the opponent tries to minimise it. In Backgammon, however, future dice rolls introduce uncertainty. Therefore, chance nodes must be included in the search tree, and the values of future positions must be weighted by the probabilities of the corresponding dice rolls.

The agent uses the evaluation function from the one-step-ahead AI to assign values to board states at the leaves of the search tree. Instead of only evaluating the immediate resulting board positions, the advanced agent searches several moves ahead. At each level of the tree, the value depends on whose turn it is and whether the next event is a player decision or a dice roll. For a decision node belonging to the AI player, the value is computed as:

$$V(s) = \max_{a \in A(s)} V(\text{Result}(s, a))$$

where $A(s)$ is the set of legal actions in state s, and $\text{Result}(s, a)$ is the board state obtained after applying action a. For a decision node belonging to the player, the value is computed as:

$$V(s) = \min_{a \in A(s)} V(\text{Result}(s, a))$$

This assumes that the player plays rationally and tries to minimise the AI opponent's expected advantage. For a chance node, the value is computed as the expected value over all possible dice outcomes:

$$V(s) = \sum_{d \in D} P(d) \cdot V(\text{Result}(s, d))$$

where D is the set of possible dice outcomes and $P(d)$ is the probability of dice outcome d.

Since two six-sided dice are used, there are 36 possible ordered dice outcomes. However, not all totals are equally likely. For example, rolling a total of 7 is more likely than rolling a total of 3, because there are more dice combinations that produce 7. The algorithm therefore weights each outcome according to its probability.

The search depth determines how many turns ahead the AI agent evaluates. A greater search depth generally leads to stronger play, because the agent can consider more long-term consequences. However, increasing the depth also increases the computational cost significantly. If there are up to 15 legal moves for a given dice roll and 36 possible dice outcomes, then even a shallow search can generate a very large number of board states. This number grows exponentially with the depth of the search tree.

For this reason, the advanced AI agent uses a fixed search depth. The agent evaluates all board states up to that depth and then applies the objective function to estimate the value of the final positions. It then selects the move that maximizes its expected value while taking into account both the opponent's best responses and the probabilities of future dice rolls.

This multi-step-ahead approach produces a more strategic opponent than the one-step-ahead AI. It is able to consider future risks, possible counter-moves, and the uncertainty caused by dice rolls. As a result, it is better suited for higher difficulty levels.

5 Potential Simplifications and Extensions

To conclude, we consider several possible extensions and simplifications of the game. These modifications can be used to adjust the overall complexity of the project depending on the available development time and the progress made during implementation. Some changes reduce the size of the game state and make the game easier to implement and test, while others introduce additional strategic depth and make the AI opponents more challenging to design.

To decrease complexity

1. **Reduce the number of checkers per player.** In standard Backgammon, each player has fifteen checkers. By reducing this number, the number of legal moves and possible board configurations decreases. This would make the game faster to simulate and would reduce the branching factor of the AI search tree.
2. **Reduce the number of points on the board.** A standard Backgammon board consists of twenty-four points. A smaller board would reduce the distance checkers need to travel and decrease the number of possible moves. This would simplify move generation and make the evaluation of board states less computationally expensive.
3. **Reduce the number of dice.** By using only one die instead of two, the number of possible dice outcomes is reduced from 36 ordered outcomes to only 6. This makes uncertainty easier to handle, since each outcome from 1 to 6 has the same probability. It also simplifies move generation, because the AI only has to consider one movement value per turn instead of combinations of two dice. In addition, special cases such as rolling doubles, where a player may make additional moves, do not need to be implemented. This reduces the branching factor of the search tree and allows the AI to search deeper within the same amount of computation time.

To increase complexity

1. **Add the doubling cube mechanic.** The doubling cube is an important feature of competitive Backgammon. It allows players to increase the stakes of the game during play. Adding this mechanic would introduce an additional strategic decision: whether to offer, accept, or reject a double. For the AI, this would require estimating not only the best move, but also the probability of winning from a given board position.
2. **Add different tournament rules.** Tournament Backgammon often includes match-based scoring, Crawford rules, and other rule variations. Adding tournament rules would allow the game to move beyond single-game decision-making. The AI would then need to consider not only the current board state, but also the current match score when making decisions.
3. **Introduce fog-of-war.** A fog-of-war mechanic could hide part of the opponent's position or restrict the information available to each player. This would move the game from a perfect-information setting to an imperfect-information setting. As a result, AI would need to make decisions based on incomplete information, which could require probabilistic reasoning or belief-state modeling.